

Emaan Wallpapers - End to End Architecture Guide (Hinglish)

Is guide ka objective hai ki tum project ko ****starting se end tak**** samjho:

- app ka startup behavior
 - auth flow
 - subscription/payment flow
 - backend edge functions
 - data model
 - har folder/file ka role
-

1) High-Level Architecture

Project 3 major layers me divide hota hai:

1. ****Presentation Layer (Android Compose UI)****

- Screens: `Splash`, `Login`, `Otp`, `ProfileSetup`, `Subscription`, `Home`, `Reels`, `Profile`
- Navigation: `AppNavGraph`
- UI Components: `CompactTopBar`, `FloatingBottomBar`, premium components

2. ****Data + Integration Layer (Android side)****

- `SupabaseClient`: DB/Storage access
- `SubscriptionApi`: Edge Functions call karna (create-order, create-subscription, activate-trial, cancel)
- `DataStoreManager`: local session/profile flags
- `SubscriptionManager`: Razorpay callback state bridge

3. ****Backend Layer (Supabase + Razorpay)****

- Edge Functions (`supabase/functions/\*`)
 - Postgres users table fields for subscription lifecycle
 - Razorpay webhooks for recurring lifecycle updates
-

2) App Startup and Navigation Flow

Entry point

- `MainActivity`:
- Compose set karta hai (`AppNavGraph`).
- Razorpay callbacks (`onPaymentSuccess`, `onPaymentError`) receive karta hai.
- Callback events ko `SubscriptionManager` me push karta hai.

Route graph

- `AppNavGraph` routes define karta hai:
 - `splash` -> `login` -> `otp` -> `profile\_setup/subscription/home`
 - `home`, `reels`, `profile`
 - `FloatingBottomBar` only selected routes pe show hota hai (`home`, `reels`).
-

3) Authentication and User Onboarding

Login

- `LoginScreen`:
- phone number validate
- OTP send (Fast2SMS / dummy path)
- OTP screen pe navigate

OTP verify

- `OtpScreen`:
- OTP verify
- DataStore me login save
- Supabase `users` row check
- decide:
- profile incomplete -> `profile\_setup`
- subscribed -> `home`
- not subscribed -> `subscription`

Profile setup

- `ProfileSetupScreen`:
- first_name etc save
- profile completion flag set

4) Subscription System (Most Important)

Core idea

Ye 2-step checkout hai, double-charge nahi:

1. **Step-1 Entry fee**: ₹5 payment via Razorpay `order\_id`
2. **Step-2 Mandate approval**: Razorpay `subscription\_id` approve AutoPay (₹99/month)
3. **Then** trial activate hota hai (3 days), then home access

Important files

- Android flow UI: `ui/subscription/SubscriptionScreen.kt`
- Checkout state manager: `ui/subscription/SubscriptionManager.kt`
- API calls: `network/SubscriptionApi.kt`
- Backend:
- `functions/create-order`
- `functions/razorpay-subscription`
- `functions/activate-trial`
- `functions/cancel-subscription`
- `functions/razorpay-webhook`

Checkout state safety

- `CheckoutKind` enum:
- `NONE`
- `ENTRY\_FEE`
- `MANDATE`
- `SubscriptionManager.prepareCheckout(kind)` checkout se pehle call hota hai.
- Kind SharedPreferences me persist hota hai so app kill/resume pe bhi state recover ho jaye.

Success handling logic

- `PaymentResult.Success` me phase detect hota hai:
- `ENTRY\_FEE`:
- `createSubscription(phone, paymentId)`
- full-screen setup loader
- ~800ms delay
- mandate checkout open
- `MANDATE`:
- `activateTrialWithRetries(phone)`
- success overlay show
- user tap Continue -> home

Duplicate protection

- `entryFeeSuccessHandled` ensure karta hai:
- `createSubscription` duplicate na chale
- callback duplicate aaye tab bhi safe flow rahe

5) Backend Edge Functions Deep Dive

1) `create-order`

- Razorpay `/v1/orders` call
- fixed amount `500` paise (₹5)
- return: `order\_id`

2) `razorpay-subscription`

- input: `phone`, `razorpay\_payment\_id`
- user existence verify
- Razorpay subscription create
- users table update:
- `razorpay\_subscription\_id`
- `subscription\_status = "created"`
- `is\_subscribed = false`
- `razorpay\_payment\_id`

3) `activate-trial`

- input: `phone`
- DB se `razorpay\_subscription\_id` fetch
- Razorpay subscription status check
- allowed statuses: `created`, `authenticated`, `active`
- DB update:
- `is\_subscribed = true`
- `subscription\_status = "trial"`
- `trial\_end` = now + 3 days

4) `cancel-subscription`

- input subscription id
- Razorpay cancel request + DB status patch

5) `razorpay-webhook`

- Razorpay HMAC verify
 - event-based DB updates:
 - `subscription.charged` -> `active`
 - `subscription.cancelled` -> `cancelled`
 - `subscription.halted/completed` -> `expired`
-

6) Data Model (users table fields)

Migration file: `supabase/migrations/20250329120000\_subscription\_system.sql`

Important columns:

- `is\_subscribed` (bool)
 - `subscription\_status` (`created` | `trial` | `active` | `cancel\_requested` | `cancelled` | `expired`)
 - `razorpay\_subscription\_id`
 - `razorpay\_payment\_id`
 - `trial\_end`
 - `trial\_paid`
 - `current\_period\_end`
 - `cancelled\_at`
 - `plan\_id`
-

7) Home and Access Control

`HomeScreen` aur `SplashScreen` dono me guard/recovery logic hai:

- agar `is\_subscribed != true`, redirect to subscription
- agar `razorpay\_subscription\_id` present but subscribed false:
- `activateTrialWithRetries` try hota hai
- successful ho to user retain in home

Isse edge case handle hota hai jaha mandate success ho gaya but DB sync delayed ho.

8) Profile Module

`ProfileScreen` me:

- user details edit
 - avatar upload via Supabase Storage
 - membership card (Trial / Active / Cancel Requested)
 - trial countdown
 - cancel subscription entry point
 - non-premium user ke liye upgrade navigation
-

9) UI/Theme Layer

Theme files:

- `theme/Theme.kt`

- `theme/Color.kt`
- `theme/Type.kt`

Reusable components:

- `CompactTopBar` (home top bar + premium badge)
- `FloatingBottomBar` (animated nav)
- `PremiumSubscriptionComponents` (premium overlays/cards/buttons)

10) File-by-File Study Order (Recommended)

Ye order follow karo, fast mastery milegi:

1. `navigation/AppNavGraph.kt`
2. `MainActivity.kt`
3. `data/DataStoreManager.kt`
4. `network/SupabaseClient.kt`
5. `ui/splash/SplashScreen.kt`
6. `ui/auth/LoginScreen.kt`
7. `ui/auth/OtpScreen.kt`
8. `ui/auth/ProfileSetupScreen.kt`
9. `ui/subscription/SubscriptionManager.kt`
10. `network/SubscriptionApi.kt`
11. `ui/subscription/SubscriptionScreen.kt`
12. `ui/subscription/PremiumSubscriptionComponents.kt`
13. `ui/subscription/SubscriptionUi.kt`
14. `ui/home/HomeScreen.kt`
15. `ui/profile/ProfileScreen.kt`
16. `ui/components/\*`
17. `supabase/functions/\*`
18. `supabase/migrations/\*`

11) End-to-End Event Timeline (Simple)

1. User app open -> Splash checks login/profile/subscription.
2. Login + OTP verify.
3. If not subscribed -> SubscriptionScreen.
4. Click `Start 5 Trial` -> create order -> Razorpay entry fee payment.
5. Success callback (`ENTRY\_FEE`) -> create subscription API.
6. Full-screen message: setup + autopay explanation.
7. Mandate Razorpay opens (`subscription\_id`).
8. Success callback (`MANDATE`) -> activate trial API.
9. Success overlay -> Continue -> Home.
10. Renewal/cancel lifecycle webhook se update hota rahta hai.

12) Revision Checklist (Quick)

Study complete tab bolo jab tum:

- `ENTRY\_FEE` aur `MANDATE` ka difference explain kar sako
- bata sako `order\_id` vs `subscription\_id` kaha use hota hai
- `SubscriptionManager` ka purpose samjha sako
- `activate-trial` status checks explain kar sako
- `is\_subscribed` false but `razorpay\_subscription\_id` present scenario samjha sako
- webhook events ka DB effect bata sako

Agar chaho to next step me isi guide ka **interview-style Q&A version** bhi bana deta hoon (questions + expected answers), jisse tum oral explanation bhi confidently de pao.